Project Planning & Management

Movie Search Application — React.js

Project	Framework	API	Team	Duration
Movie Search App	React.js (SPA)	Movie Database API (To Be Selected)	5 Members	4 Weeks

1. Project Proposal

The Movie Search Application is a React-based Single Page Application (SPA). It lets users search for movies and view details for each one. The app connects to an external Movie Database API to get movie data like titles, posters, ratings, and descriptions.

Project Info

Field	Details
Project Name	Movie Search Application
Type	Academic Web Development Project
Framework	React.js — Single Page Application
API	Movie Database API (To Be Selected)
Team Size	5 Members — all React Developers
Duration	4 Weeks
Methodology	Agile — 4 one-week sprints

Objectives

- Build a React SPA that connects to an external Movie Database API
- Build a Search Bar that shows movie results from the API
- Build a Movie Details page showing full info for each movie
- Use React best practices: functional components, hooks, and props
- Make the UI responsive and clean on both desktop and mobile

Scope

In Scope:

- Search Bar — search movies by keyword using the API
- Results Page — grid of movie cards with title, poster, year
- Details Page — full info: plot, director, actors, ratings
- Responsive design for desktop and mobile
- Loading and error states using React state management

Out of Scope:

- No backend or database — frontend only
- No user login or watchlists
- No video streaming

Team Members

ID	Name	Role
M1	Alyaa Mosad Aly Ezz	Project Lead + API Integration
M2	Mohamed Shabaan Suliman Elgishy	Search Feature Developer
M3	Nourhan Sameh Gamal Fayad	UI / Styling Developer
M4	Mohanad Mohamed Fekry Mohamed	Results and Cards Developer
M5	Ahmed Hassan Ahmed Khalil	Details Page Developer

2. Project Plan

The project is divided into 4 weekly sprints. The table below shows the planned tasks and which sprint each one is scheduled in.

Gantt Chart

#	Task	Who	Sp.1	Sp.2	Sp.3	Sp.4
1	GitHub repo setup + API research	Alyaa	■			
2	Wireframes + component tree design	Nourhan	■			
3	Write all project documents	Alyaa	■			
4	App layout + client-side routing	Mohamed S.		■		
5	Search Bar + API integration	Mohamed S.		■		
6	Results Grid + Movie Cards	Mohanad		■		
7	Movie Details page	Ahmed H.			■	
8	Pagination + Load More	Mohanad			■	
9	Loading and error states	Alyaa			■	
10	CSS styling + responsive layout	Nourhan			■	
11	Testing and bug fixes	All				■
12	Final review + submission	All				■

Milestones

Sprint	Milestone	Owner
Sprint 1	GitHub repo created, API selected, wireframes done	Alyaa
Sprint 2	Search bar works with real API data, results shown on screen	Mohamed S.
Sprint 3	Details page complete, all pages responsive on mobile	Ahmed H.
Sprint 4	All tests passed, all docs submitted, demo ready	All

Tech Stack

Technology	Purpose
React.js + Vite	Main framework for building the SPA
React Router	Client-side navigation between pages
Movie Database API	Source of all movie data (to be selected)
CSS Modules	Styling — scoped per component
GitHub	Version control and team collaboration

3. Task Assignment & Roles

All 5 team members are React developers. Each person is responsible for a specific part of the app. No work has started yet — this is the planned division.

Team Roles

ID	Name	Responsibilities
M1	Alyaa Mosad Aly Ezz	Sprint planning, GitHub setup, API integration, error handling
M2	Mohamed Shabaan Suliman Elgishy	Search Bar component, API call, search results state
M3	Nourhan Sameh Gamal Fayad	Wireframes, CSS Modules, responsive layout, visual design
M4	Mohanad Mohamed Fekry Mohamed	Movie Card component, Results Grid, pagination
M5	Ahmed Hassan Ahmed Khalil	Movie Details page, fetch movie by ID, display all fields

Task List

ID	Task	Assigned To	Sprint
T01	Initialize React project with Vite	Alyaa	Sprint 1
T02	Setup GitHub repo and branching strategy	Alyaa	Sprint 1
T03	Research and select the Movie Database API	Alyaa	Sprint 1

T04	Write all project planning documents	Alyaa	Sprint 1
T05	Design wireframes and component tree	Nourhan	Sprint 1
T06	Setup client-side routing and page structure	Mohamed S.	Sprint 2
T07	Build Search Bar component	Mohamed S.	Sprint 2
T08	Integrate Movie Database API for search	Alyaa	Sprint 2
T09	Build Movie Card component	Mohanad	Sprint 2
T10	Build Results Grid with movie cards	Mohanad	Sprint 2
T11	CSS styling for Search page	Nourhan	Sprint 2
T12	Build Movie Details page component	Ahmed H.	Sprint 3
T13	Fetch and display full movie data on Details page	Ahmed H.	Sprint 3
T14	Implement Load More and pagination	Mohanad	Sprint 3
T15	Loading and error state UI components	Alyaa	Sprint 3
T16	CSS styling for Details page and mobile	Nourhan	Sprint 3
T17	Cross-browser and mobile testing	All Team	Sprint 4
T18	Bug fixes and code cleanup	All Team	Sprint 4
T19	Final documentation and submission	Alyaa	Sprint 4

4. Risk Assessment & Mitigation Plan

This section lists the risks we expect to face during the project and how we plan to handle them. No risk has occurred yet.

Risk Register

ID	Risk	Prob.	Impact	Mitigation
R01	API has usage limits or needs a paid plan	Medium	Medium	Research free-tier APIs in Sprint 1 and identify a backup
R02	API returns no results for some searches	High	Low	Show a friendly empty-state message to the user
R03	API key accidentally exposed in code	Medium	High	Store key in .env file and add it to .gitignore from day one
R04	Git merge conflicts between team members	High	Medium	Use feature branches and merge via Pull Requests only
R05	Scope creep — adding unplanned features	High	Medium	Freeze scope after Sprint 1, any change needs team approval
R06	React state becoming too complex	Medium	Medium	Keep state local where possible, share only when needed
R07	Mobile responsiveness issues	Low	Medium	Use mobile-first CSS and test on different

				screen sizes each sprint
R08	Team member unfamiliar with React	Medium	Medium	Knowledge-sharing in Sprint 1, lead reviews all code before merging
R09	Slow API responses	Low	Low	Show a loading spinner while data is being fetched
R10	Project delayed past deadline	Medium	High	Reserve Sprint 4 for testing only — no new features in last sprint

Contingency Plan

- Weekly standup to track risks and blockers each sprint
- All code pushed to GitHub regularly to avoid data loss
- If the API is down during the demo, use a local mock data file
- If a team member falls behind, tasks are redistributed in the next standup

5. Key Performance Indicators (KPIs)

These are the metrics we will use to measure if the project is successful. All targets below are goals — no measurement has been taken yet.

Performance

ID	Metric	Target	How to Measure
KPI-P01	API response time	Less than 2 seconds	Chrome DevTools — Network tab
KPI-P02	Search results render time	Less than 3 seconds	Chrome Lighthouse audit
KPI-P03	Details page load time	Less than 2 seconds	Chrome Lighthouse audit

Quality

ID	Metric	Target	How to Measure
KPI-Q01	Search accuracy	10/10 test queries return correct results	Manual test with fixed queries
KPI-Q02	Error handling	5/5 error scenarios handled without crash	Test: empty input, bad query, timeout, no image, invalid ID
KPI-Q03	Mobile responsiveness	Works on 375px, 414px, 768px screens	Chrome DevTools device emulator
KPI-Q04	Cross-browser compatibility	Works on Chrome, Firefox, and Edge	Manual QA testing

Delivery & User Experience

ID	Metric	Target
KPI-D01	Milestones delivered on time	At least 3 of 4 milestones on time
KPI-D02	Features completed	100% — Search and Details both working
KPI-D03	Documentation submitted	All 5 documents submitted
KPI-U01	Search task completion time	User can search a movie in under 30 seconds
KPI-U02	Empty state UX	No blank screens — message always shown when no results
KPI-U03	Navigation clarity	Back button always visible and working

KPI Summary

Category	Count	When to Evaluate
Performance	3	End of each sprint during testing
Quality	4	End of Sprint 3 and Sprint 4
Delivery	3	End of each sprint
User Experience	3	Sprint 4 — before final submission
Total	13	—